COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	July 24, 2026	**DATE SUBMITTED:**	July 31, 2026
NAME:	Uchi, Mao Sophie S.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 **Linked Lists**

1. PROCEDURE OUTPUT

Table 3-1. Output of Initial/Simple Implementation

Screenshot

main.cpp HOA3P1 ● main.cpp SW3 ×

HOA3P1 > main.cpp > ...

```
#include<iostream>
#include<utility>

class Node{
public:
    char data;
    Node *next;
};

int main(){
    //step 1
    Node *head = NULL;
    Node *second = NULL;
    Node *third = NULL;
    Node *fourth = NULL;
    Node *fifth = NULL;
    Node *last = NULL;

    //step 2
    head = new Node;
    second = new Node;
    third = new Node;
    fourth = new Node;
    fifth = new Node;
    last = new Node;

    //step 3
    head->data = 'C';
    head->next = second;

    second->data = 'P';
    second->next = third;

    third->data = 'E';
    third->next = fourth;

    fourth->data = '0';
    fourth->next = fifth;

    fifth->data = '1';
    fifth->next = last;

    //step 4
    last->data = '0';
    last->next = nullptr;
}
```

Discussion
The first step shown in our main program is to declare 6 individual named pointers such as ‘head’, ‘second’, ‘third’, ‘fourth’, ‘fifth’ and ‘last’ to a node and initialize them to NULL. The second step is shown to use the New Node to allocate the memory from each node on the heap. The third step is shown containing each node with specific char characters to point to the New Node. Lastly the fourth step is shown the ‘last’ named node to be initialized with the character ‘0’ and continue next to a nullptr to end the list. From the steps created, you’ve successfully completed the element word CPE010.

Table 3-2. Code for the List Operations

Operation	Screenshot
Traversal	

```
26  template <typename T>
27  void dllTraverse(Doublylist<T>* currentNode){
28      //check if the list is empty
29      if(currentNode == nullptr){
30          std::cout << "The list is empty";
31          return;
32      }
33
34      Doublylist<T>* tail;
35
36      std::cout<<"FORWARD"<<std::endl;
37      //forward traverse
38      while(currentNode != nullptr){
39          std::cout << currentNode->data << " ";
40          tail = currentNode;
41          currentNode=currentNode->next;
42      }
43      std::cout<<std::endl;
44      //reverse traverse
45      std::cout<<"REVERSE"<<std::endl;
46      while(tail != nullptr){
47          std::cout<<tail->data<<" ";
48          tail= tail->prev;
49      }
50      std::cout<<std::endl;
51
52  }
```

Insertion at head

```
54  //inserting a node to the head
55  template <typename T>
56  void dllInsertHead(T newData, Doublylist<T>** currentNode){
57
58      // create a new node:
59      Doublylist<T> *newNode = createNode(newData);
60      //point the newNode to the currentNode (head)
61      newNode->next = *currentNode;
62      //point the currentNode(head) to the newNode
63      (*currentNode)->prev = newNode;
64      //updating the header pointer
65      *currentNode = newNode;
66  }
```

Insertion at any part of the list

```
85  //General inserting
86  template <typename T>
87  void dllGeneralInsert(T newData, Doublylist<T> *currentNode) {
88
89      Doublylist<T>* newNode = createNode(newData);
90
91      newNode->next = currentNode->next;
92      newNode->prev = currentNode;
93
94      if(currentNode->next != nullptr) {
95          currentNode->next->prev = newNode;
96      }
97
98      currentNode->next = newNode;
99  }
```

Insertion at the end

```
68  //inserting at the End
69  template <typename T>
70  void dllInsertEnd(T newData, Doublylist<T> *currentHead) {
71      // create a new node
72      Doublylist<T>* newNode = createNode(newData);
73
74      //traverse until we reach the last node
75      while(currentHead->next != nullptr) {
76          currentHead = currentHead->next;
77      }
78
79      //connect the new node back to the last node
80      newNode->prev = currentHead;
81      //connect the current node to the new node
82      currentHead->next = newNode;
83  }
```

Deletion of a node

```
101  //Deleting Node
102  template <typename T>
103  void dllDeleteNode(T findData, Doublylist<T>** head)
104  {
105      if(*head == nullptr) return;
106
107      Doublylist<T>* currNode =  *head;
108      while (currNode->next != nullptr) {
109          currNode = currNode->next;
110      }
111
112      while(currNode != nullptr && currNode->data != findData){
113          currNode = currNode->prev;
114      }
115
116      if(currNode == nullptr) return;
117
118      if(currNode->prev != nullptr){
119          currNode->prev->next = currNode->next;
120      }
121      else {
122          *head = currNode->next;
123      }
124
125      if (currNode->next != nullptr) {
126          currNode->next->prev = currNode->prev;
127      }
128
129      delete currNode;
130
131  }
```

Table 3-3. Code and Analysis for Singly Linked Lists (Doubly Linked Lists)

a

Source Code

table3p2.cpp > main()

```cpp
#include <iostream>
#include "doubly_ll.h"

int main(){

    // C P E 0 1 0
    Doublylist<char>* head = createNode('C');
    Doublylist<char>* second = createNode('P');
    Doublylist<char>* third = createNode('E');
    Doublylist<char>* fourth = createNode('0');
    Doublylist<char>* fifth = createNode('1');
    Doublylist<char>* sixth = createNode('0');

    //connecting the nodes:
    head->next = second;
    second->prev = head;
    second->next = third;
    third->prev = second;
    //connecting 3 more nodes:
    third->next = fourth;
    fourth->prev = third;
    fourth->next = fifth;
    fifth->prev = fourth;
    fifth->next = sixth;
    sixth->prev = fifth;

    std::cout << "Testing the dllTraverse List:" <<std::endl;
    dllTraverse(head);
```

Console

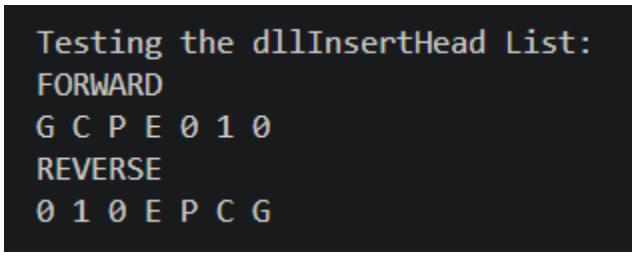
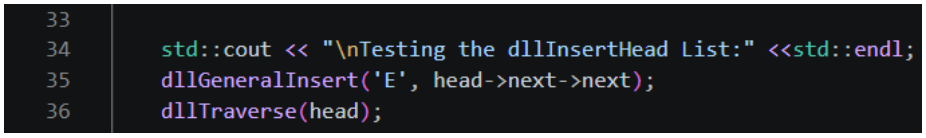
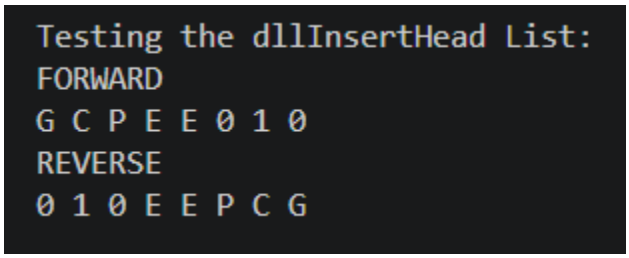
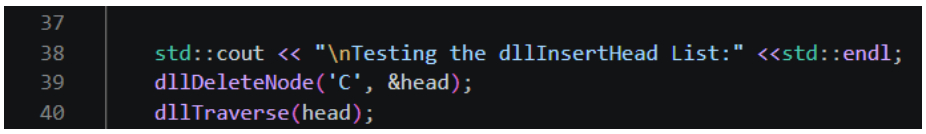
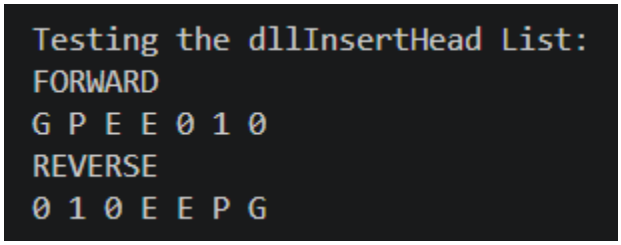
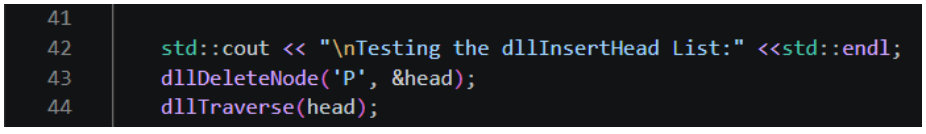
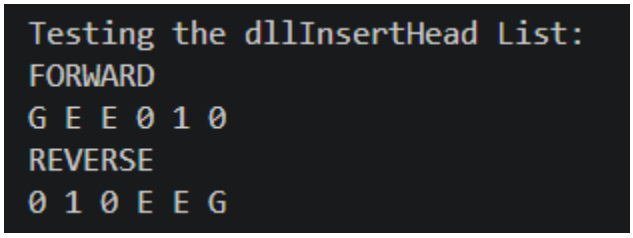
```
Testing the dllTraverse List:
FORWARD
C P E 0 1 0
REVERSE
0 1 0 E P C
```

b

Source Code

```cpp

    std::cout << "\nTesting the dllInsertHead List:" <<std::endl;
    dllInsertHead('G', &head);
    dllTraverse(head);
```

	Console	
c	**Source Code**	
	Console	
d	**Source Code**	
	Console	
e	**Source Code**	
	Console	

f	**Source Code**	45 46 `std::cout << "\nTesting the dllTraverse List:" <<std::endl;` 47 `dllTraverse(head);`

	Console	`Testing the dllTraverse List:`
		`FORWARD`
		`G E E 0 1 0`
		`REVERSE`
		`0 1 0 E E G`
		`PS D:\Downloads (HDD)\CPE010\HOA3P1\Hoa3p1>`

Table 3-4. Modified Operations for Doubly Linked Lists

Screenshots(s)	Analysis
26 `template <typename T>` 27 `void dllTraverse(Doublylist<T>* currentNode){` 28 `//check if the list is empty` 29 `if(currentNode == nullptr){` 30 `std::cout << "The list is empty";` 31 `return;` 32 `}` 33 34 `Doublylist<T>* tail;` 35 36 `std::cout<<"FORWARD"<<std::endl;` 37 `//forward traverse` 38 `while(currentNode != nullptr){` 39 `std::cout << currentNode->data << " ";` 40 `tail = currentNode;` 41 `currentNode=currentNode->next;` 42 `}` 43 `std::cout<<std::endl;` 44 `//reverse traverse` 45 `std::cout<<"REVERSE"<<std::endl;` 46 `while(tail != nullptr){` 47 `std::cout<<tail->data<<" ";` 48 `tail= tail->prev;` 49 `}` 50 `std::cout<<std::endl;` 51 52 `}`	Traversal: A Traversal Doubly List prints out every node beginning from 'head' node and ending from an 'end' node from the list in a chronological order of pushed data which is called Forward Traverse AND reverse chronology order with the recent pushed data called Reverse Traverse. The difference would be for Singly and Doubly Trasversal List is Singly can only Traverse forward as the nodes are linked to a 'next' node, while Doubly can Traverse both forward and reverse(backward) using the linked nodes 'next' and 'prev'(previous node).

```cpp
//inserting a node to the head
template <typename T>
void dllInsertHead(T newData, Doublylist<T>** currentNode){

    // create a new node:
    Doublylist<T> *newNode = createNode(newData);
    //point the newNode to the currentNode (head)
    newNode->next = *currentNode;
    //point the currentNode(head) to the newNode
    (*currentNode)->prev = newNode;
    //updating the header pointer
    *currentNode = newNode;
}
```

Insert Head:

An Insert Head Doubly List inserts a new node head, also known as the first node, pushing the linked nodes and giving space to the new node in front of the other nodes, like cutting off a line to pass everyone else who's already been there. For reverse Insert Head, from a mirrored point of view, the head would not be replaced, instead pulling in the linked list to create space for the 'end' node.

```cpp
//General inserting
template <typename T>
void dllGeneralInsert(T newData, Doublylist<T> *currentNode) {

    Doublylist<T>* newNode = createNode(newData);

    newNode->next = currentNode->next;
    newNode->prev = currentNode;

    if(currentNode->next != nullptr) {
        currentNode->next->prev = newNode;
    }

    currentNode->next = newNode;
}
```

General Insert:

A General Insert Doubly List is similar to both Insert Head and Insert End, but all of the nodes should pull out from their perspective position to create space for the new node. The new node can be placed anywhere and does not need any exact position to be placed.

```cpp
//inserting at the End
template <typename T>
void dllInsertEnd(T newData, Doublylist<T> *currentHead) {
    // create a new node
    Doublylist<T>* newNode = createNode(newData);

    //traverse until we reach the last node
    while(currentHead->next != nullptr) {
        currentHead = currentHead->next;
    }

    //connect the new node back to the last node
    newNode->prev = currentHead;
    //connect the current node to the new node
    currentHead->next = newNode;
}
```

Insert End:

An Insert End Doubly List inserts a new node end, also known as the last node, pulling the linked nodes from the left and giving space to the new node at the end, like patiently waiting from a line and getting ready to be moved whenever. For reverse Insert End, from a mirrored point of view, the end would not be replaced, instead pushing out the linked list to create space for the 'head' node.

```cpp
//Deleting Node
template <typename T>
void dllDeleteNode(T findData, Doublylist<T>** head)
{
    if(*head == nullptr) return;

    Doublylist<T>* currNode =  *head;
    while (currNode->next != nullptr) {
        currNode = currNode->next;
    }

    while(currNode != nullptr && currNode->data != findData){
        currNode = currNode->prev;
    }

    if(currNode == nullptr) return;

    if(currNode->prev != nullptr){
        currNode->prev->next = currNode->next;
    }
    else {
        *head = currNode->next;
    }

    if (currNode->next != nullptr) {
        currNode->next->prev = currNode->prev;
    }

    delete currNode;

}
```

Delete Node: (add more maybe?)

A Delete Node Doubly List (NOT DeleteList) deletes a specific node, creating an empty space to be pulled in by the previous linked nodes to occupy the vacant node.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++

Create a playlist using multiple songs.

mainPlaylist.cpp > main()

```cpp
#include "Playlist.h"
#include <iostream>

int main() {
    Song *dllplaylistHead = nullptr;

    std::cout << "\n^v^ CURRENT PLAYLIST ^v^" << std::endl;
    dllInsertHeadPlaylist("Numb by Linkin Park", &dllplaylistHead);
    dllInsertHeadPlaylist("Free Bird by Lynyrd Skynyrd", &dllplaylistHead);
    dllInsertHeadPlaylist("In The End by Linkin Park", &dllplaylistHead);
    dllInsertHeadPlaylist("Smells Like Teen Spirit by Nirvana", &dllplaylistHead);
    dllInsertHeadPlaylist("Chop Suey! by System Of A Down", &dllplaylistHead);
    dllTraversePlaylist(dllplaylistHead);

```

Add songs to the playlist.

```
14
15      std::cout << "ADDED A SONG!\n" << std::endl;
16      dllInsertHeadPlaylist("Somebody Told Me by The Killers", &dllplaylistHead);
17      dllTraversePlaylist(dllplaylistHead);
18
```

Remove a song from the playlist.

```
19      std::cout << "...REMOVING A SONG...\n" << std::endl;
20      dllDeleteNodePlaylist("Somebody Told Me by The Killers", &dllplaylistHead);
21      dllTraversePlaylist(dllplaylistHead);
```

Play songs in a loop (for this activity, we will print all the songs once).

```
23      std::cout << "\n^v^ PLAYLIST ^v^" << std::endl;
24      Song *currentSong = dllplaylistHead;
25
26      if (currentSong != nullptr) {
27          std::cout << "Current Song: " << currentSong->data << std::endl;
28          currentSong = currentSong->next;
29      }
30      if (currentSong != nullptr) {
31          std::cout << "Next Song: " << currentSong->data << std::endl;
32          currentSong = currentSong->prev;
33      }
34      if (currentSong != nullptr) {
35          std::cout << "Previous Song: " << currentSong->data << std::endl;
36          std::cout << "\n";
37      }
38
39      return 0;
```

Playlist.h

Playlist.h > dllTraversePlaylist(Song *)

```
#ifndef PLAYLIST_H
#define PLAYLIST_H

#include <iostream>
#include <string>

class Song{
    public:
    std::string data;
    Song* next = nullptr;
    Song* prev = nullptr;
};

Song* dllcreatingSong(std::string newData){
    Song *newSong = new Song;
    newSong->data = newData;
    newSong->prev = newSong;
    newSong->next = newSong;
    return newSong;
}

void dllTraversePlaylist(Song *head) {

    if (head == nullptr) {
        std::cout << "No song in your Playlist :(" << std::endl;
        return;
    }

    Song *current = head;
    int trackNumber = 1;

    std::cout << "  Currently Playing: " << std::endl;

    do {
        std::cout << trackNumber << ". " << current->data << std::endl;
        current = current->next;
        trackNumber++;
    } while (current != head);

    std::cout << std::endl;
}
```

```
42
43  void dllInsertHeadPlaylist(std::string newData, Song** currentHead){
44      Song *newSong = dllcreatingSong(newData);
45
46      if (*currentHead == nullptr) {
47          *currentHead = newSong;
48          return;
49      }
50      Song *tail = (*currentHead)->prev;
51
52      newSong->next = *currentHead;
53      newSong->prev = tail;
54      (*currentHead)->prev = newSong;
55      tail->next = newSong;
56      *currentHead = newSong;
57  }
58
```

```
58
59  void dllDeleteNodePlaylist(std::string targetSong, Song** headRef) {
60      if (*headRef == nullptr) return;
61
62      Song* current = *headRef;
63
64      do {
65          if (current->data == targetSong) {
66              // Case 1: Only one SONG(Node) in the list
67              if(current->next == current) {
68                  *headRef = nullptr;
69              }
70              // Case 2: Multiple SONGS(Nodes) in the list
71              else {
72                  current->prev->next = current->next;
73                  current->next->prev = current->prev;
74
75                  // If deleting the head SONG(node), move SONG(node) pointer forward
76                  if (current == *headRef) {
77                      *headRef = current->next;
78                  }
79              }
80
81              delete current;
82              return;
83          }
84          current = current->next;
85      } while (current != *headRef);
86
87      std::cout << "Song unavailable in your Playlist :(" << std::endl;
88  }
89
90  #endif
```

Output

```
^v^ CURRENT PLAYLIST ^v^
  Currently Playing:
1. Chop Suey! by System Of A Down
2. Smells Like Teen Spirit by Nirvana
3. In The End by Linkin Park
4. Free Bird by Lynyrd Skynyrd
5. Numb by Linkin Park

ADDED A SONG!

  Currently Playing:
1. Somebody Told Me by The Killers
2. Chop Suey! by System Of A Down
3. Smells Like Teen Spirit by Nirvana
4. In The End by Linkin Park
5. Free Bird by Lynyrd Skynyrd
6. Numb by Linkin Park

...REMOVING A SONG...

  Currently Playing:
1. Chop Suey! by System Of A Down
2. Smells Like Teen Spirit by Nirvana
3. In The End by Linkin Park
4. Free Bird by Lynyrd Skynyrd
5. Numb by Linkin Park


^v^ PLAYLIST ^v^
Current Song: Chop Suey! by System Of A Down
Next Song: Smells Like Teen Spirit by Nirvana
Previous Song: Chop Suey! by System Of A Down
```

Conclusion
What I've concluded for this activity is, learning about the importance of singly and doubly files to coding, since both are similar in a way they function the same, but the only difference is Doubly List has an additional function that also prints out the Reverse list, the opposite of Forward Traverse. Speaking of linked lists, 5 list operations are shown to us, first is Traversal List prints each node from the list in chronological order of pushed data, starting from the "head" node and ending with the "end" node. By pushing the linked nodes and creating room for the new node in front of the other nodes, an Insert Head Doubly List inserts a new node head, also referred to as the first node. Similar to Insert Head and Insert End, a General Insert Doubly List requires each node to pull out of its perspective position in order to make room for the new node. By removing the linked nodes from the left and making room for the new node at the end, an Insert End Doubly List adds a new node end, also referred to as the final node. When a particular node is deleted using a Delete Node Doubly List, an empty space is created that can be filled by previously linked nodes. I hope these list operations would help me in my future activities, making it easier to work with a bundle of objects at once, similar to array, but now a bigger tool to check our progress if it's going smoothly.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Ai Overview, Quillbot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code Assistance, summarization

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: They're used for correction and paraphrased my explanation for better understanding

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.